Práctica 01-RSN - DSAN - Proxmox y LXC desde CLI

Práctica DSAN

14/10/2026

Proxmox y LXC desde CLI

Curso de especialización en Servicios en la Nube — Módulo de despliegue

Documento de calentamiento antes de la práctica *"Despliegue automatizado con OpenTofu en Proxmox"*. Primero haremos todo **a mano** desde la terminal del nodo Proxmox, y luego veremos que **cada cosa que hemos tecleado se convertirá en una línea de un fichero de plantilla** que OpenTofu ejecutará por nosotros.

Contents

Proxmox y LXC desde CLI 1

Objetivos 2

Situarse en Proxmox desde la terminal 3
- La API: donde todo converge . . . 3

Contenedores LXC: el ciclo de vida con `pct` 4
- Preparar la plantilla (imagen base) . . . 4
- Crear un contenedor . . . 4
- Arrancar, entrar y operar . . . 4
- Modificar en caliente, parar, destruir . . . 5

CheatSheet de `pct` 6

De la terminal a la IaC: el mapa mental 7
- ¿Por qué molestarse en cambiar a IaC si ya sabemos usar `pct`? . . . 7

Mini-ejercicios de comprobación 7

Vocabulario para el cuaderno 7

Objetivos

La progresión que vamos a seguir:



Figure 1: OpenTofu

Al terminar debéis ser capaces de responder:

- ¿Qué es un contenedor LXC y en qué se diferencia de una VM?
- ¿Cómo se crea, arranca, para y destruye un contenedor con `pct`?
- ¿Qué es la **infraestructura como código (IaC)** y qué problema resuelve?
- ¿Por qué un despliegue declarativo es un **trabajo por lotes repetible sin supervisión**?

Situarse en Proxmox desde la terminal

La web de administración de Proxmox es solo una fachada: **todo lo que hace la GUI se puede hacer desde la terminal del nodo**, y eso es justo lo que aprovechará OpenTofu después (hablando con la misma API).

```
# Información general del nodo

# version de Proxmox y de todos sus componentes
pveversion -v

# estado del cluster - nodo unico: standalone-
pvecm status

# rendimiento del almacenamiento - FIO resumido-
pveperf

# Almacenamiento disponible -
# conectad esto con el RAID que habeis montado

# lista de datastores y su uso
pvesm status

# contenido del datastore 'local'
pvesm list local

# vuestro RAID visto como sistema de ficheros
df -h /dev/md127

# Red del nodo

# interfaces y sus IPs
ip -br addr
# configuracion persistente del bridge vmbr0
cat /etc/network/interfaces
```

La API: donde todo converge

```
# El comando 'pvesh' se comunica directamente con la API de Proxmox
pvesh get /version
pvesh get /nodes

# lista de contenedores del nodo
pvesh get /nodes/$(hostname)/lxc
```

Cuando ejecutáis `pct list`, por debajo Proxmox llama a esa misma API. **OpenTofu también usará esa API** con un token (lo veremos en la práctica): por eso es importante entender que la terminal y la GUI son solo dos formas de hablar con el mismo motor.

Contenedores LXC: el ciclo de vida con `pct`

LXC no es virtualización completa: el contenedor **comparte el kernel del host** pero tiene su propio sistema de ficheros, procesos y red. Arranca en segundos y gasta muchos menos recursos que una VM — perfecto para servicios web y para hacer pruebas.

Preparar la plantilla (imagen base)

```
pveam update                        # actualizar catálogo de plantillas
pveam available | grep ubuntu       # buscar plantillas de Ubuntu
pveam download local ubuntu-26.04-standard_26.04-xx_amd64.tar.zst
pveam list local                    # comprobar que ya está descargada
```

> **Concepto IaC nº 1 → imagen base:** desplegar siempre desde una *plantilla* homogénea es el primer paso hacia el despliegue repetible. En OpenTofu, esa plantilla será una cadena en vuestro `main.tf` (`operating_system { template_id = ... }`).

Crear un contenedor

```
pct create 201 local:vztmpl/ubuntu-24.04-standard_24.04-xx_amd64.tar.zst \
  --hostname webarbolado-01 \
  --cores 2 --memory 1024 --swap 512 \
  --rootfs local-lvm:8 \
  --net0 name=eth0,bridge=vmbr0,ip=10.0.2.10/24,gw=10.0.2.1 \
  --unprivileged 1 \
  --start 0

pct list
```

Qué significa cada cosa:

Opción	Significado	Su equivalente en OpenTofu
`201`	ID numérico del contenedor	`vm_id = 200`
`local:vztmpl/...`	plantilla base	`operating_system { template_id }`
`--hostname`	nombre del contenedor	`initialization { hostname }`
`--cores 2`	CPU asignada	`cpu { cores = 2 }`
`--memory 1024`	RAM en MB	`memory { dedicated = 1024 }`
`--rootfs local-lvm:8`	disco de 8 GB en ese datastore	`disk { datastore_id, size }`
`--net0 ...`	interfaz de red y IP fija	`network_interface` + `ip_config`
`--unprivileged 1`	contenedor sin privilegios (seguridad)	`unprivileged = true`

> **Concepto IaC nº 2 — el estado deseado:** con `pct create` habéis descrito *cómo debe ser* el contenedor (CPU, RAM, red…) en una sola orden. OpenTofu lleva esta idea al extremo: describís el estado deseado en un fichero y el motor lo hace realidad y lo **mantiene**.

Arrancar, entrar y operar

```
pct start 201
pct status 201

# Ejecutar comandos DENTRO del contenedor sin SSH

# shell interactivo dentro del CT
pct exec 201 -- bash
pct exec 201 -- apt update
pct exec 201 -- systemctl status ssh

# Entrar como si fuera una consola (exit para salir)
pct enter 201
```

```
#
# Migrar en vivo / clonar / convertir en plantilla
#
# a otro nodo --- si hubiera clúster _
pct migrate 201 pve2
pct clone 201 202 --hostname webarbolado-02

# 201 deja de arrancar: es imagen base ahora
pct template 201
```

Concepto IaC nº 3 — aprovisionamiento: `pct exec` es vuestra puerta para instalar software dentro del contenedor. En la práctica, el script `setup.sh` usará exactamente este comando (vía SSH desde el equipo de gestión) para instalar nginx, el backup y el cron. Cambiará *quién* lo ejecuta (OpenTofu en vez de vosotros), pero no *el cómo*.

Modificar en caliente, parar, destruir

```
# cambiar recursos
pct set 201 --cores 4

# ampliar disco
pct resize 201 rootfs +2G

# reinicio clasico
pct shutdown 201 && pct start 201

# apagado brusco
pct stop 201

# borrar todo
pct destroy 201 --destroy-unreferenced-disks 1 --purge 1
```

Concepto IaC nº 4 — idempotencia: `pct set 201 --cores 4` podéis ejecutarlo cien veces: el resultado es siempre el mismo. Esa es la propiedad que hace posible la automatización sin supervisión: reintentar no rompe nada. OpenTofu aplica este principio a todo el despliegue completo.

Concepto IaC nº 5 — ciclo de vida completo: crear → modificar → destruir. Con la terminal, cada cambio es una orden que teclea una persona. Con IaC, **todo el ciclo** está descrito de antemano y se ejecuta con `tofu plan` / `tofu apply` / `tofu destroy` — repetible, auditable y sin intervención humana.

CheatSheet de `pct`

Comando	Función
`pct list`	listar contenedores y su estado
`pct create <id> <plantilla> [opciones]`	crear contenedor
`pct start/stop/shutdown <id>`	encender / parar / apagar limpio
`pct status <id>`	estado actual
`pct config <id>`	ver toda la configuración del CT
`pct set <id> [opciones]`	modificar configuración
`pct exec <id> -- <cmd>`	ejecutar comando dentro del CT
`pct enter <id>`	consola interactiva dentro del CT
`pct push <id> origen destino`	copiar un fichero del host al CT
`pct pull <id> origen destino`	copiar un fichero del CT al host
`pct resize <id> rootfs +2G`	ampliar disco
`pct clone <id> <nuevo-id>`	clonar
`pct destroy <id> --purge`	eliminar

Extras útiles del host:

Comando	Función
`pct list | pct fstrim <id>`	liberar espacio no usado
`lxc-info -n <id>`	detalle del contenedor (paquete lxc)
`cat /etc/pve/lxc/<id>.conf`	¡vuestra config es un fichero de texto!

Momento clave: abrid `cat /etc/pve/lxc/201.conf` y echarle un vistazo. **La configuración del contenedor ya es texto.** Proxmox la guarda en `/etc/pve` (un sistema de ficheros distribuido en memoria). La IaC no inventa nada raro: simplemente pasa de *"escribir el fichero a mano"* a *"describirlo en un lenguaje declarativo y dejar que un motor (OpenTofu) lo mantenga y lo versionado en Git"*.

De la terminal a la IaC: el mapa mental

Todo lo que habéis hecho tiene traducción directa a la práctica con OpenTofu:

Hoy (manual)	En la práctica (IaC)
`pveam download local ubuntu-...`	Plantilla ya disponible; se referencia en `main.tf`
`pct create 201 ... --cores 2 --memory 1024`	Recurso `proxmox_virtual_environment_container` con `cpu {}` y `memory {}`
`pct set 201 --cores 4`	Editar `main.tf` + `tofu plan` (muestra el delta) + `tofu apply`
`pct exec 201 -- apt install nginx`	Provisioner `local-exec` que lanza `setup.sh` vía `pct exec`
`cat /etc/pve/lxc/201.conf`	`tofu state list` / `tofu state show`
`pct destroy 201`	`tofu destroy`
Vosotros tecleando cada orden	**Trabajo por lotes**: nadie teclea nada, se ejecuta repetitivo y sin supervisión

¿Por qué molestarse en cambiar a IaC si ya sabemos usar `pct`?

1. **Repetibilidad:** desplegar 3 contenedores idénticos con `pct` es copiar/pegar comandos; con IaC es `count = 3`.
2. **Sin supervisión:** `tofu apply` se puede ejecutar desde un cron o un pipeline CI sin nadie delante y nosotros tomando café. — exactamente el RA del módulo.
3. **Versionado:** el `main.tf` vive en Git: cada cambio de infraestructura tiene autor, fecha y *diff*.
4. **Drift:** `tofu plan` compara el estado real con el deseado y avisa si alguien ha "tocado" algo a mano por la GUI… como `pct set`.
5. **Destruir y recrear es barato:** si el despliegue está descrito, la infraestructura se vuelve desechable.

Mini-ejercicios de comprobación

Esto debe costar alrededor de 10 min, sin mirar apuntes.

1. Cread un contenedor `301` llamado `prueba-lxc` con 1 núcleo, 512 MB de RAM, 4 GB de disco e IP `10.0.2.100/24`. Arrancadlo y verificad con `pct exec 301 -- ip addr`.
2. Cambiadle la RAM a 1024 MB sin destruirlo (`pct set`). Comprobadlo con `pct config 301`.
3. Instalad `nginx` dentro con `pct exec` y haced `curl` a su IP desde vuestro portátil.
4. Copiad un fichero cualquiera del host al contenedor con `pct push`.
5. Mirad `/etc/pve/lxc/301.conf` y **identificad cada línea** con la opción de `pct create` que la generó.
6. Destruidlo todo (`pct destroy 301`).

(Cuando terminéis: en la práctica de OpenTofu repetiréis los pasos 1–6 del tirón con un solo `tofu apply` y un solo `tofu destroy`.)

Vocabulario para el cuaderno

- **LXC / contenedor:** aislamiento a nivel de SO compartiendo kernel del host.
- **Plantilla (`vztmpl`):** imagen base comprimida de un sistema para crear contenedores.
- **Datastore:** almacenamiento de Proxmox donde viven plantillas, discos y backups (`local`, `local-lvm`, vuestro RAID…).
- **Contenedor sin privilegios (unprivileged):** aislamiento mediante user namespaces; siempre que se pueda, la opción por defecto.
- **IaC (Infraestructura como Código):** describir la infraestructura en ficheros versionados y ejecutables.
- **Estado deseado vs. estado real:** lo que vuestro código pide vs. lo que existe; OpenTofu reconcilia ambos.
- **Idempotencia:** reejecutar produce siempre el mismo resultado.
- **Drift:** desviación entre estado real y estado deseado (alguien cambió algo a mano).
- **Trabajo por lotes:** secuencia de tareas ejecutada de forma repetitiva y sin supervisión directa.